

JavaFX Cheat Sheet - Examen MAP

Acest ghid acoperă controalele esențiale cerute în examen (TableView, ComboBox, DatePicker etc.) și modul de interacțiune cu ele.

1. Structura de Bază (Controller)

Majoritatea logicii se află în metoda initialize.

```
public class MyController {
    @FXML private Button btnSave;
    @FXML private Label lblMessage;

    // Metoda apelată automat după încărcarea fișierului FXML
    @FXML
    public void initialize() {
        // Inițializări, populare tabele, listeners
    }

    // Legat din FXML prin onAction="#handleSave"
    @FXML
    public void handleSave(ActionEvent event) {
        System.out.println("Buton apăsăat!");
    }
}
```

2. TableView (Esențial)

Tabelul este cea mai complexă componentă. Necesită maparea coloanelor la atributele clasei de model.

Proprietăți Cheie:

- items: Lista observabilă de date.
- selectionModel: Gestionează rândul selectat.

Setup & Populare:

```
@FXML private TableView<Student> tableView;
@FXML private TableColumn<Student, String> colNume;
@FXML private TableColumn<Student, Double> colMedie;

private ObservableList<Student> model = FXCollections.observableArrayList();
```

```

@FXML
public void initialize() {
    // 1. Mapare Coloane (String-ul trebuie să coincidă cu numele getter-ului: "nume" ->
    getNume())
    colNume.setCellValueFactory(new PropertyValueFactory<>("nume"));
    colMedie.setCellValueFactory(new PropertyValueFactory<>("medie"));

    // 2. Setare Date
    tableView.setItems(model);

    // 3. Ascultare Selecție (Când dai click pe un rând)
    tableView.getSelectionModel().selectedItemProperty().addListener((obs, oldVal, newVal) ->
    {
        if (newVal != null) {
            System.out.println("Selectat: " + newVal.getNume());
            fillFormFields(newVal); // Populează formularul de editare
        }
    });
}

```

Custom TableRows (Stilizare condiționată):

Util dacă vrei să colorezi rânduri în funcție de o condiție (ex: nota < 5).

```

tableView.setRowFactory(tv -> new TableRow<Student>() {
    @Override
    protected void updateItem(Student item, boolean empty) {
        super.updateItem(item, empty);
        if (item == null || empty) {
            setStyle("");
        } else {
            // Condiție pentru stilizare
            if (item.getMedie() < 5) {
                setStyle("-fx-background-color: #ffcccc;"); // Roșu deschis pentru restanță
            } else if (item.getMedie() >= 9.5) {
                setStyle("-fx-background-color: #ccffcc;"); // Verde pentru bursă
            } else {
                setStyle(""); // Resetare stil default
            }
        }
    }
}
}

```

```
});
```

Operații Utile:

```
// Obține elementul selectat curent
Student s = tableView.getSelectionModel().getSelectedItem();

// Refresh forțat (dacă modifici obiectul dar tabelul nu se actualizează)
tableView.refresh();
```

3. ListView (Listă Simplă)

Similar cu TableView, dar afișează o singură coloană/celulă per rând. Este util pentru liste simple de obiecte.

Setup & Populare:

```
@FXML private ListView<Student> listView;
private ObservableList<Student> listModel = FXCollections.observableArrayList();

@FXML
public void initialize() {
    listView.setItems(listModel);

    // Randare Personalizată (Custom Cell Factory)
    // Implicit ListView apelează toString(). Aici definim noi ce să afișeze.
    listView.setCellFactory(param -> new ListCell<Student>() {
        @Override
        protected void updateItem(Student item, boolean empty) {
            super.updateItem(item, empty);

            if (empty || item == null) {
                setText(null);
            } else {
                setText(item.getNum() + " - Medie: " + item.getMedie());
                // Putem seta și stiluri:
                // setStyle("-fx-font-weight: bold;");
            }
        }
    });
```

```
// Ascultare Selecție
listView.getSelectionModel().selectedItemProperty().addListener(obs -> {
    System.out.println("List item clicked");
});
}
```

4. ComboBox (Liste Derulante)

Folosit pentru filtrare sau selectarea unei opțiuni dintr-un set fix (ex: Enum).

Proprietăți Cheie:

- items: Lista de opțiuni.
- value: Elementul selectat curent.

Cod Util:

```
@FXML private ComboBox<String> comboFilter;
```

```
// 1. Populare (cu String-uri sau Enum-uri)
```

```
comboFilter.setItems(FXCollections.observableArrayList("Optiune A", "Optiune B"));
```

```
// SAU cu Enum:
```

```
// comboFilter.setItems(FXCollections.observableArrayList(StareCivila.values()));
```

```
// 2. Selectare programatică
```

```
comboFilter.getSelectionModel().selectFirst();
```

```
// 3. Obținere valoare
```

```
String val = comboFilter.getValue();
```

```
// 4. Eveniment la schimbare (util pentru filtrare instantanee)
```

```
comboFilter.setOnAction(event -> {
    handleFilter(comboFilter.getValue());
});
```

5. TextField & TextArea

Pentru input de text.

Proprietăți Cheie:

- text: Conținutul textului.
- editable: Dacă se poate scrie în el.

Cod Util:

```
@FXML private TextField txtSearch;

// 1. Get / Set
String text = txtSearch.getText();
txtSearch.setText("Valoare nouă");
txtSearch.clear();

// 2. Search în timp real (Listener pe proprietatea text)
txtSearch.textProperty().addListener((observable, oldValue, newValue) -> {
    filterListByName(newValue);
});
```

6. Controale de Selecție (CheckBox, Radio, Date)

CheckBox (Bifă Simplă)

```
@FXML private CheckBox chkActive;

// Verificare
if (chkActive.isSelected()) { ... }

// Setare
chkActive.setSelected(true);
```

RadioButton (Exclusivitate)

Important: Trebuie adăugate într-un ToggleGroup pentru a funcționa corect (doar unul selectat la un moment dat).

```
@FXML private RadioButton rbMale, rbFemale;
private ToggleGroup group = new ToggleGroup();

@FXML public void initialize() {
    rbMale.setToggleGroup(group);
    rbFemale.setToggleGroup(group);

    // Hack util: Setare UserData pentru a recupera valoarea ușor
    rbMale.setUserData("M");
    rbFemale.setUserData("F");
}
```

```
// Obținere valoare selectată
RadioButton selected = (RadioButton) group.getSelectedToggle();
if (selected != null) {
    String val = (String) selected.getUserData();
}
```

DatePicker (Calendar)

Lucrează cu LocalDate.

```
@FXML private DatePicker datePicker;

// 1. Obținere dată
LocalDate date = datePicker.getValue();

// 2. Conversie pt Baza de Date (SQL)
java.sql.Date sqlDate = java.sql.Date.valueOf(date);

// 3. Setare dată (ex: azi)
datePicker.setValue(LocalDate.now());
```

7. Feedback Utilizator (Alerts)

Nu folosi System.out.println pentru erori la examen! Folosește Alert.

```
private void showMessage(Alert.AlertType type, String header, String content) {
    Alert alert = new Alert(type);
    alert.setHeaderText(header);
    alert.setContentText(content);
    alert.showAndWait();
}

// Utilizare:
showMessage(Alert.AlertType.ERROR, "Eroare DB", "Nu s-a putut salva!");
showMessage(Alert.AlertType.INFORMATION, "Succes", "Date salvate.");
```

8. Responsive UI (Multithreading)

Dacă folosești Service asincron (cu CompletableFuture), **NU** ai voie să modifci UI-ul de pe

thread-ul de background.

Regula de Aur: Orice `setText`, `setItems`, `add` în UI trebuie făcut în `Platform.runLater`.

```
service.findAllAsync().thenAccept(data -> {  
    // Background Thread -> NU atinge UI-ul aici!  
  
    Platform.runLater(() -> {  
        // Application Thread -> Aici e sigur  
        model.setAll(data);  
        loadingSpinner.setVisible(false);  
    });  
});
```

9. Proprietăți Comune & Bindings (Trucuri)

Toate controalele (Node) au aceste proprietăți utile:

- `setVisible(boolean)`: Ascunde/Arată elementul.
- `setDisable(boolean)`: Dezactivează interacțiunea (devine gri).
- `setStyle(String)`: CSS inline (ex: `-fx-background-color: red;`).

Binding (Clean Code):

Dezactivează butonul "Add" automat dacă textul e gol, fără if-else.

```
// În initialize():  
btnAdd.disableProperty().bind(txtNume.textProperty().isEmpty());
```

10. Ferestre Noi (Stages & Navigation)

Deschiderea unei noi ferestre (ex: pentru Editare/Adăugare) implică încărcarea unui FXML nou și crearea unui nou Stage.

Cod Boilerplate:

```
public void handleOpenNewWindow(ActionEvent event) {  
    try {  
        // 1. Încărcare FXML  
        FXMLLoader loader = new FXMLLoader(getClass().getResource("EditView.fxml"));  
        Parent root = loader.load();  
  
        // 2. Transmitere Date către noul Controller (CRUCIAL!)
```

```

EditController ctrl = loader.getController();
ctrl.setService(this.service); // Injectare Service
// ctrl.setStudent(selectedStudent); // Dacă e fereastră de editare

// 3. Creare Stage (Fereastră)
Stage stage = new Stage();
stage.setTitle("Editare Student");
stage.setScene(new Scene(root));

// 4. Afișare
// stage.show(); // Non-modal (poți da click și în spate)
stage.showAndWait(); // Modal (blochează fereastră veche până se închide asta)

} catch (IOException e) {
    e.printStackTrace();
}
}

```

11. Controale în Tabel (Butoane, CheckBox, ComboBox)

Pentru a adăuga elemente interactive în celulele tabelului (ex: buton "Delete" pe fiecare rând), trebuie definit un `CellFactory`.

A. Adăugarea unui Buton pe Coloană

Util pentru ștergere rapidă sau acțiuni per rând.

```

// Definire în FXML: <TableColumn fx:id="colAction" text="Actiuni"/>
@FXML private TableColumn<Student, Void> colAction;

@FXML
public void initialize() {
    // ... setup alte coloane ...

    colAction.setCellFactory(param -> new TableCell<>() {
        // Creăm butonul o singură dată per celulă
        private final Button btn = new Button("Delete");

        {
            // Acțiunea butonului
            btn.setOnAction(event -> {

```



```

        // Obținem obiectul de pe rândul curent
        Student student = getTableView().getItems().get(getIndex());
        System.out.println("Sterge studentul: " + student.getNum());
        // service.delete(student.getId());
    });
}

@Override
protected void updateItem(Void item, boolean empty) {
    super.updateItem(item, empty);
    if (empty) {
        setGraphic(null); // Dacă rândul e gol, nu afișa nimic
    } else {
        setGraphic(btn); // Altfel, afișează butonul
    }
}
});
}

```

B. ComboBox în Tabel (Editare Inline)

Permite modificarea unei valori direct din tabel. Tabelul trebuie să fie editabil (`editable="true"` în FXML).

```

@FXML private TableView<Student> tableView;
@FXML private TableColumn<Student, String> colStare; // ex: "ACTIV", "INACTIV"

@FXML public void initialize() {
    tableView.setEditable(true);

    // Folosim o implementare standard din JavaFX
    colStare.setCellFactory(ComboBoxTableCell.forTableColumn("ACTIV", "INACTIV",
        "SUSPENDAT"));

    // Salvăm modificarea când utilizatorul alege altceva
    colStare.setOnEditCommit(event -> {
        Student s = event.getRowValue();
        s.setStare(event.getNewValue());
        System.out.println("Stare modificată în: " + event.getNewValue());
    });
}

```

C. CheckBox în Tabel (Boolean)

Dacă obiectul are un câmp Boolean sau BooleanProperty.

```
@FXML private TableColumn<Student, Boolean> colPrezent;
```

```
@FXML public void initialize() {  
    // Folosim implementarea standard CheckBoxTableCell  
    // Nota: coloana trebuie să fie legată de un BooleanProperty în model pentru a fi reactivă  
    automat  
    colPrezent.setCellFactory(CheckBoxTableCell.forTableColumn(colPrezent));
```

```
  
    // SAU Varianta manuală (pentru control total):  
    /*  
    colPrezent.setCellFactory(col -> new TableCell<>() {  
        private final CheckBox chk = new CheckBox();  
        {  
            chk.setOnAction(e -> {  
                Student s = getTableView().getItems().get(getIndex());  
                s.setPrezent(chk.isSelected());  
            });  
        }  
        @Override  
        protected void updateItem(Boolean item, boolean empty) {  
            super.updateItem(item, empty);  
            if (empty || item == null) {  
                setGraphic(null);  
            } else {  
                chk.setSelected(item);  
                setGraphic(chk);  
            }  
        }  
    });  
    */  
}
```